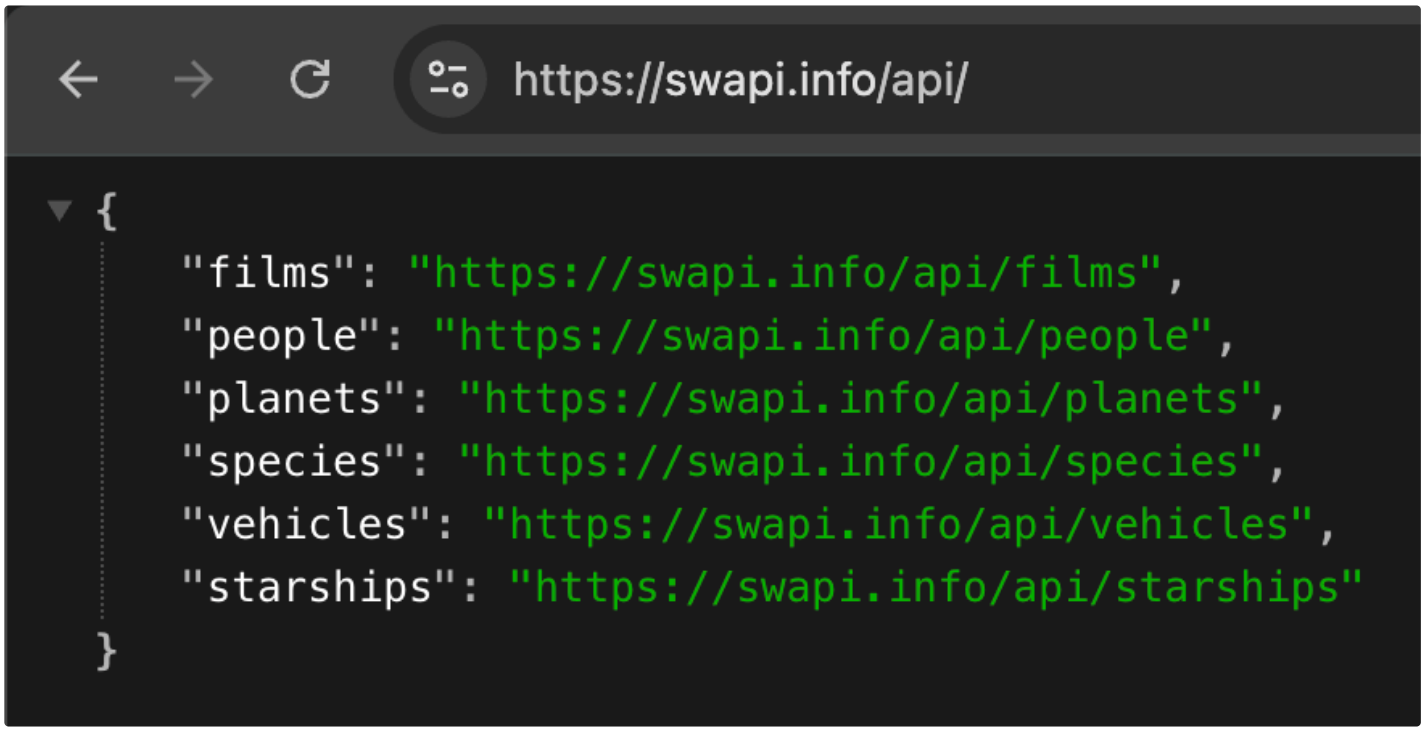
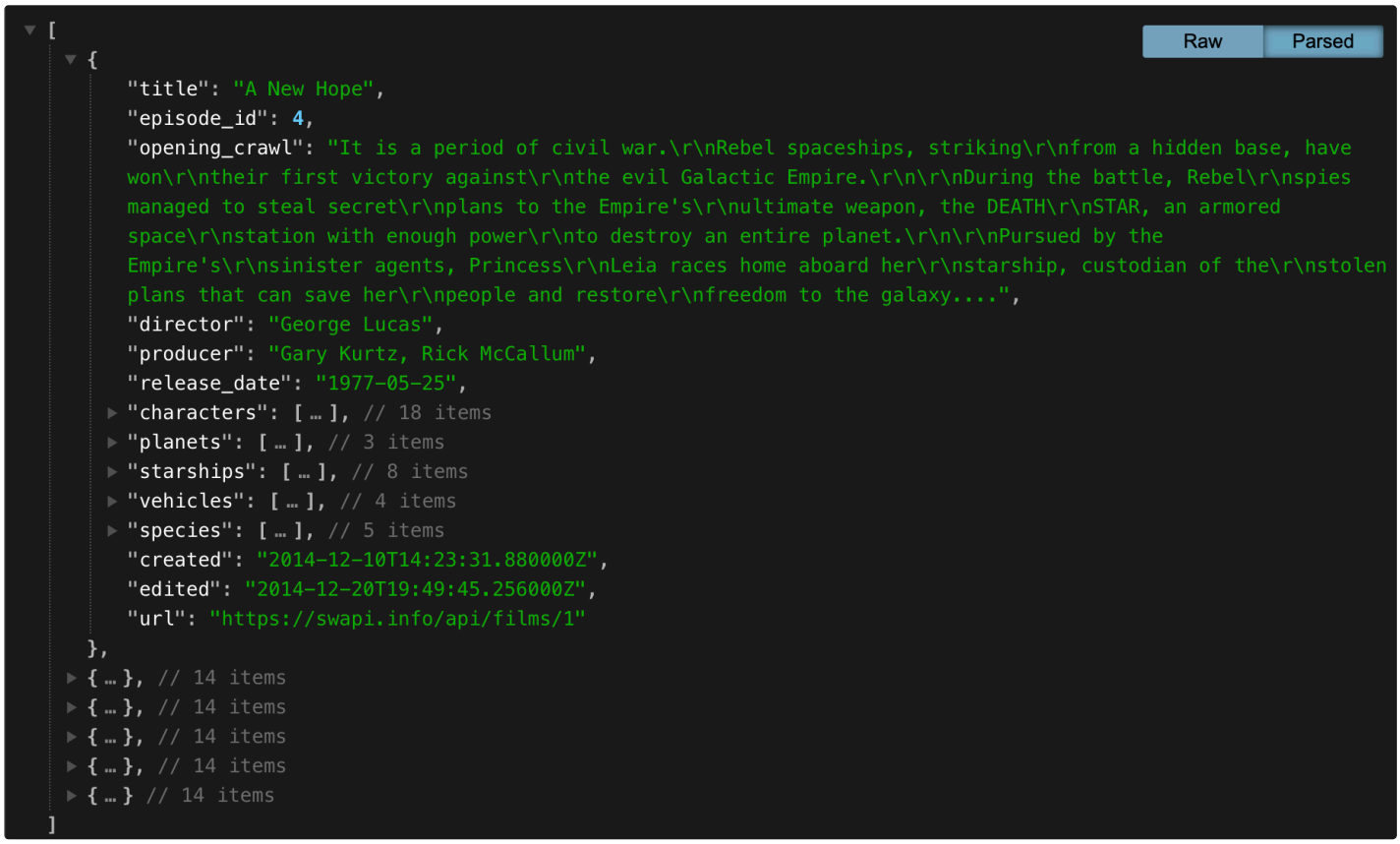
Assignment_5

Task1(a):

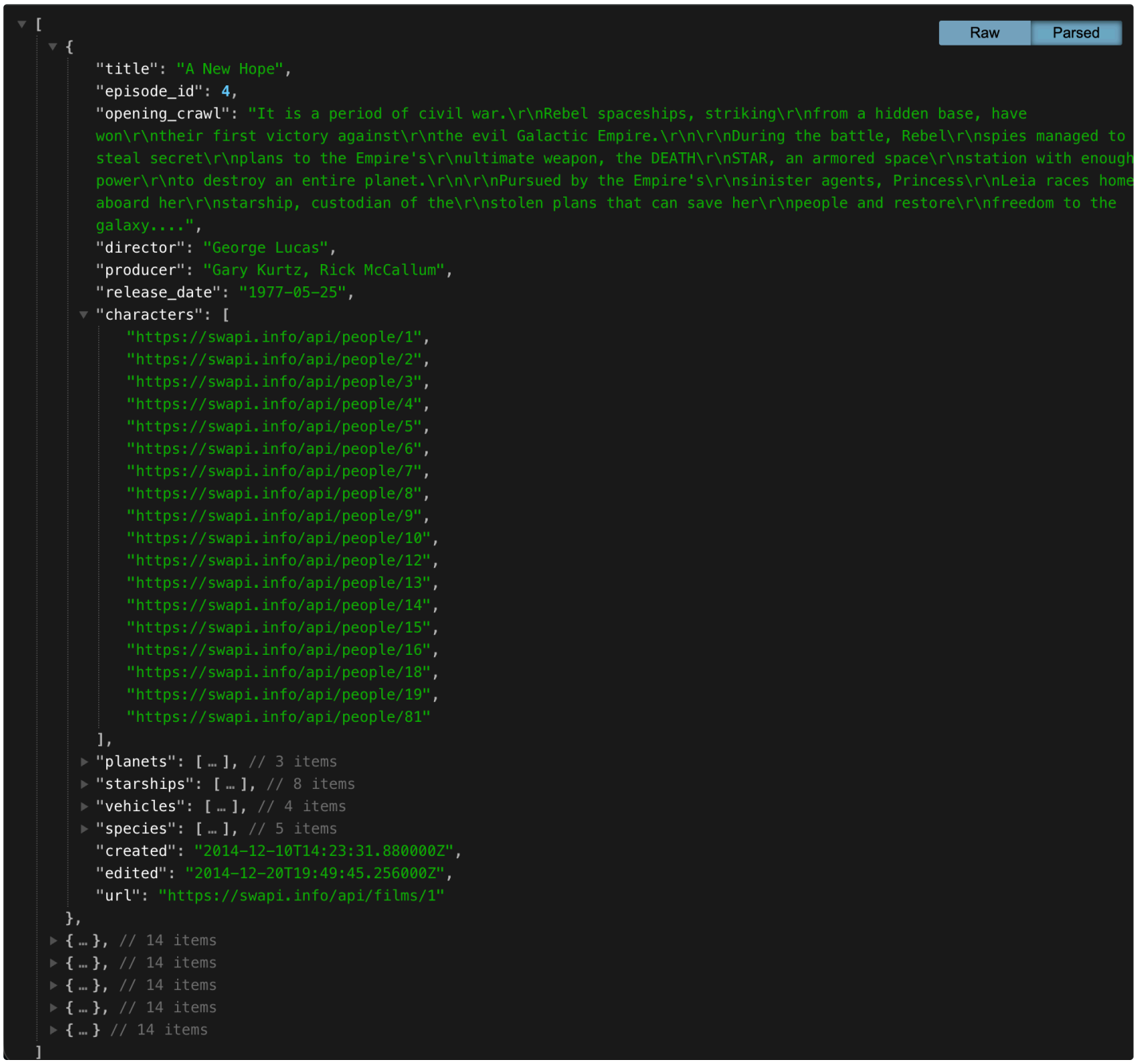


When hitting the "https://swapi.info/api/" API endpoint, we get a JSON response containing a list of all the available API endpoints : films, people, planets, species, vehicles and starships.

1. The first endpoint "https://swapi.info/api/films":



When calling the "films" endpoint we get a list of all the 6 movies, in each one of them there are scalar fields such as 'title' 'episode_id' and some reference fields, each one of them holds a list of API endpoints representing a relationship between the film and the field. As an example, the field 'characters' represents the 'people' that appears in that specific movie.



When expanding the 'characters' field, we find each person in the film represented as an API endpoint with the specific id of that person, and in order to get the data of it, you need to call again that specific endpoint.

scalar fields	**reference fields**

• title • episode_id • director • producer • release_date • created • edited • url	• characters • planets • starships • vehicles • species

> *We categorised 'url' as a scalar field not reference despite having it an API endpoint because it doesn't represent a relationship or connecting a different entity, it points to the object itself.*

2. The API endpoint "https://swapi.info/api/people":


Raw
Parsed
[
{
"name": "Luke Skywalker",
"height": "172",
"mass": "77",
"hair_color": "blond",
"skin_color": "fair",
"eye_color": "blue",
"birth_year": "19BBY",
"gender": "male",
"homeworld": "https://swapi.info/api/planets/1",
"films": [
"https://swapi.info/api/films/1",
"https://swapi.info/api/films/2",
"https://swapi.info/api/films/3",
"https://swapi.info/api/films/6"
],
"species": [],
"vehicles": [
"https://swapi.info/api/vehicles/14",
"https://swapi.info/api/vehicles/30"
],
"starships": [
"https://swapi.info/api/starships/12",
"https://swapi.info/api/starships/22"
],
"created": "2014-12-09T13:50:51.644000Z",
"edited": "2014-12-20T21:17:56.891000Z",
"url": "https://swapi.info/api/people/1"
},
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items
{ … }, // 16 items

For the second endpoint we have the same as the first one, some scalar fields alongside some reference one to connect the data.

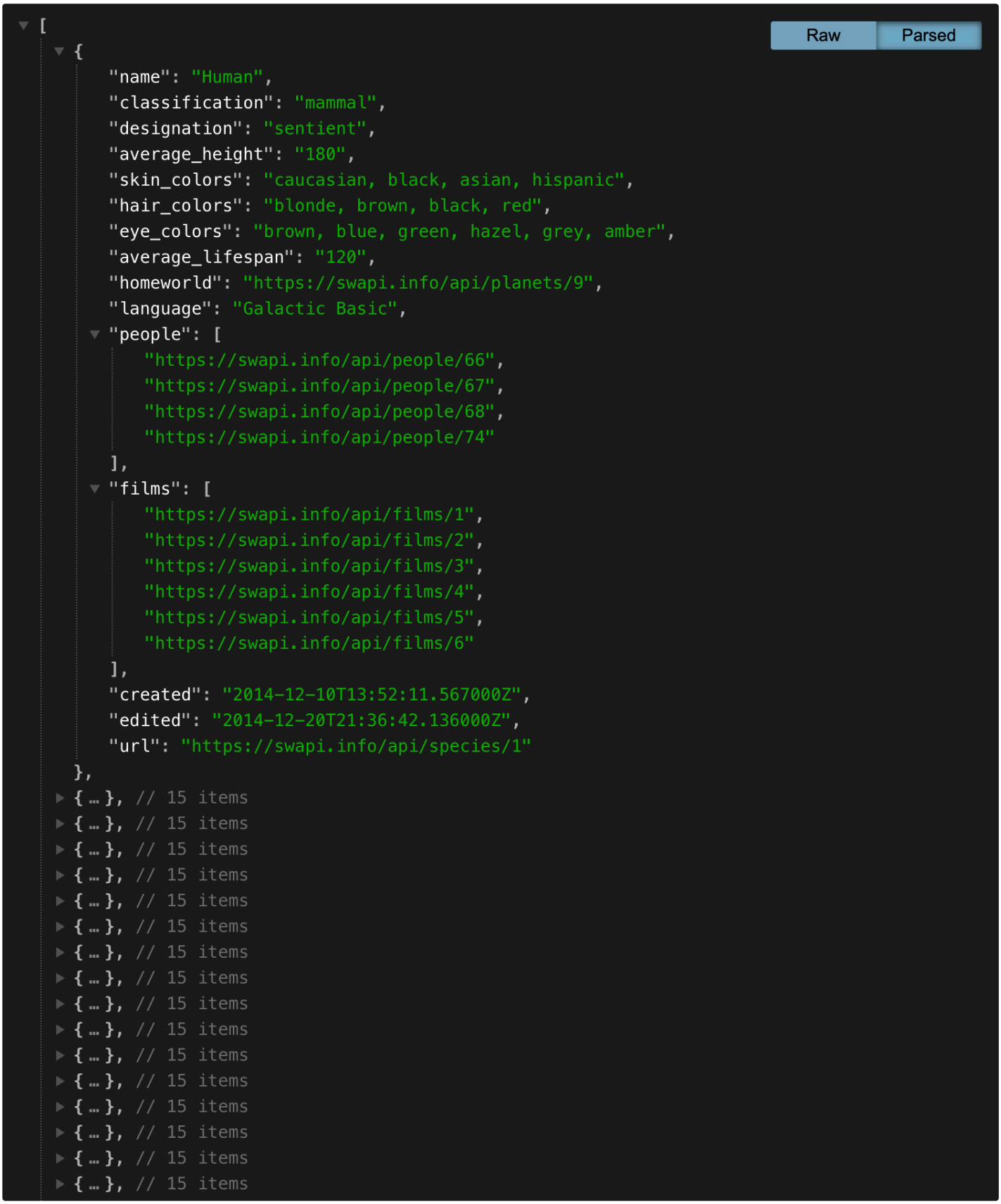
scalar fields	**reference fields**
• name • height • mass • hair_color • skin_color • eye_color • birth_year • gender • created • edited • url	• homeworld • films • species • vehicles • starships

3. The API endpoint : "https://swapi.info/api/planets":

scalar fields	reference fields
• name • rotation_period	• residents • films

• orbital_period • diameter • climate • gravity • terrain • surface_water • population • created • edited • url	

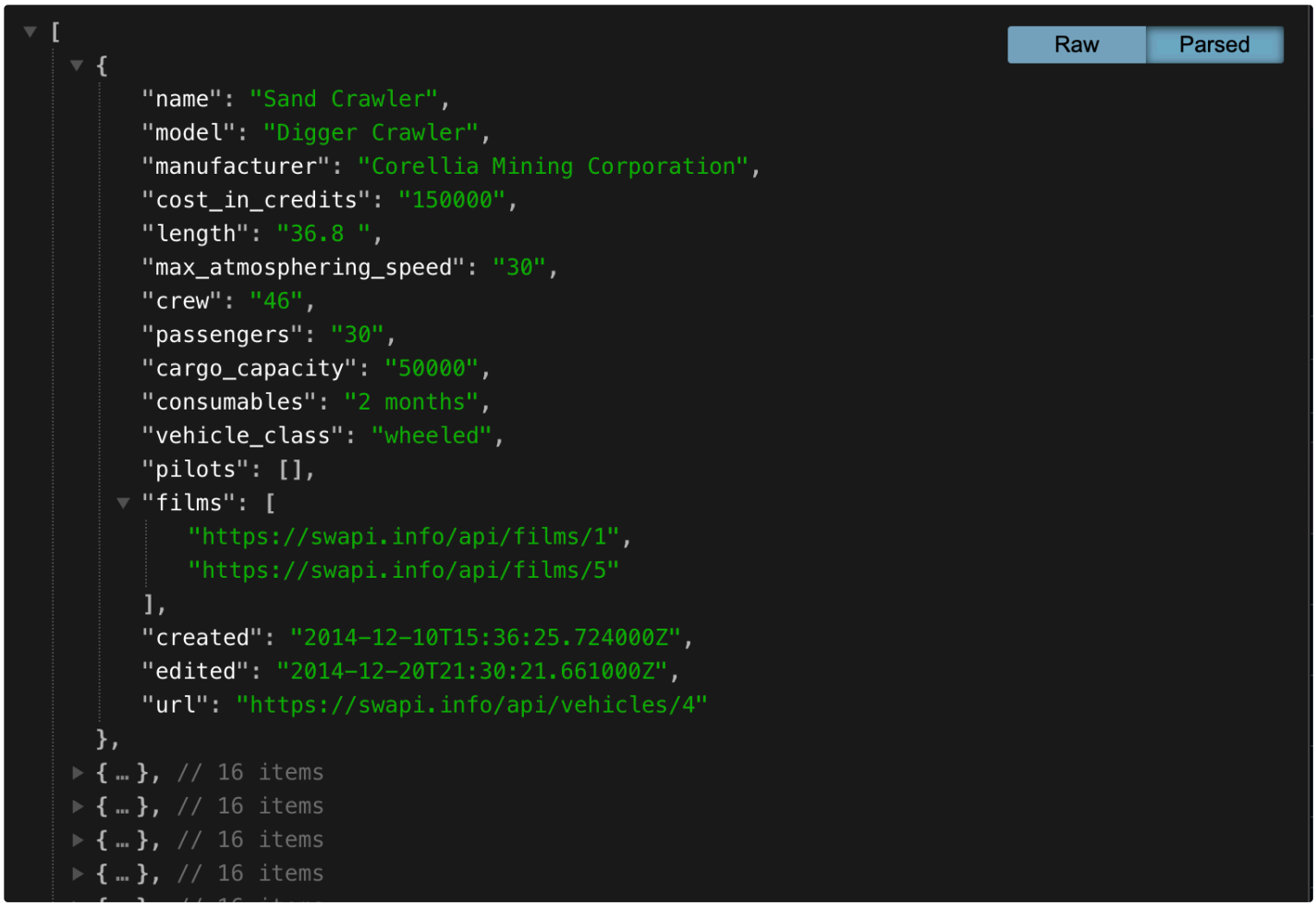
4. The API endpoint "https://swapi.info/api/species":



scalar fields	reference fields

• name • classification • designation • average_height • skin_colors • hair_colors • eye_colors • average_lifespan • language • created • edited • url	• homeworld • people • films

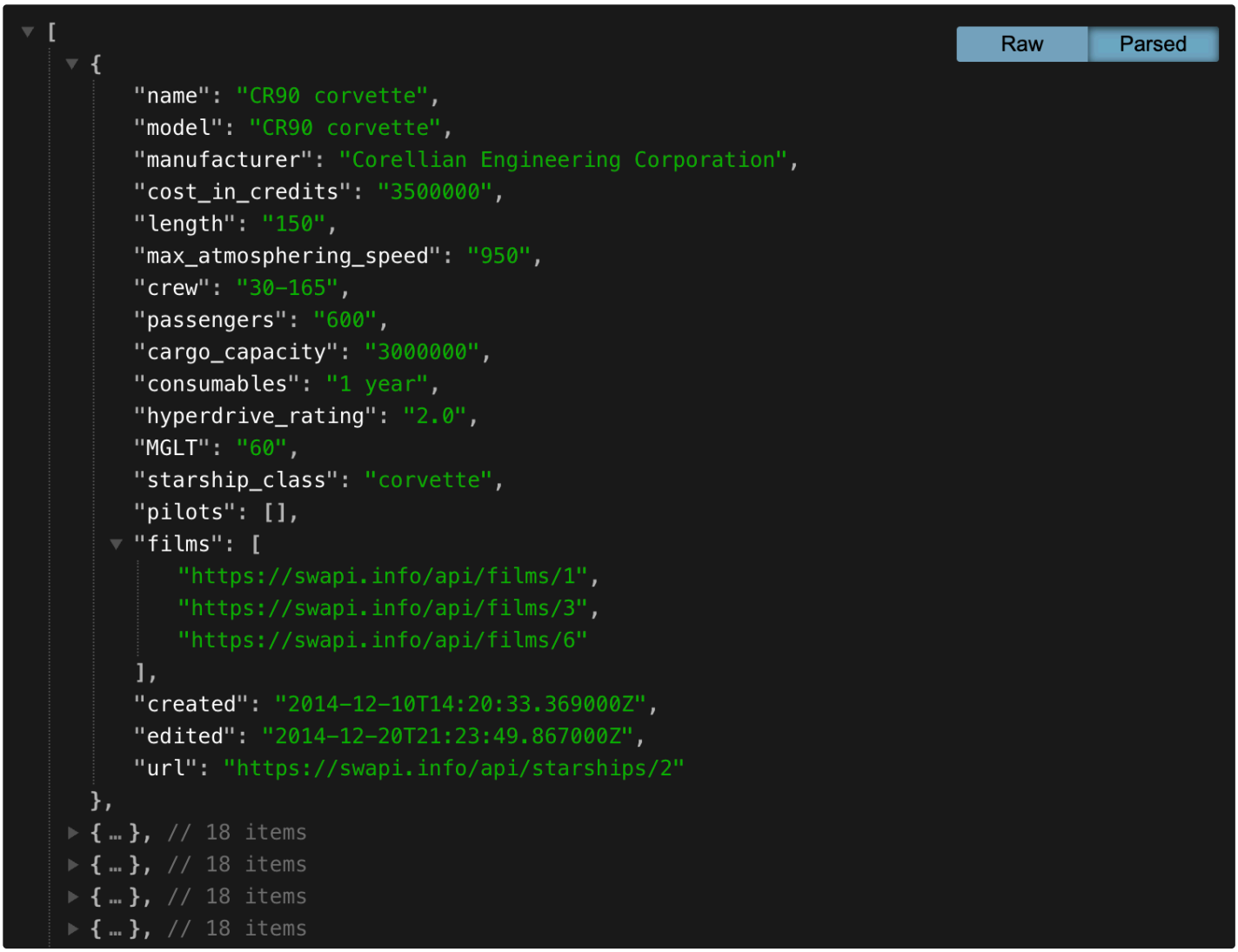
5. The API endpoint "https://swapi.info/api/vehicles" :



scalar fields	reference fields
• name • model • manufacturer • cost_in_credits • length • max_atmosphering_speed	• pilots • films

• crew • passengers • cargo_capacity • consumables • vehicle_class • created • edited • url	

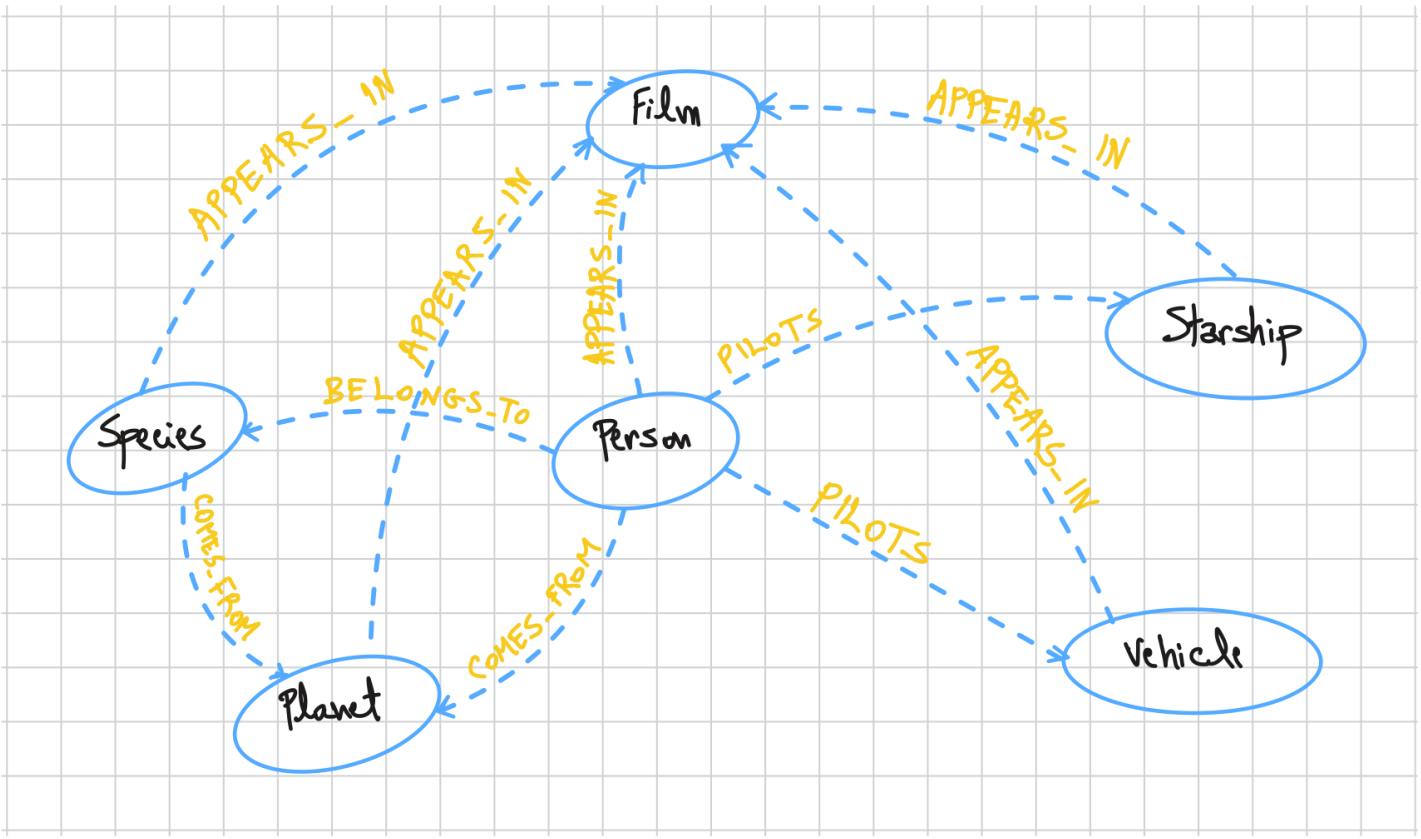
6. The API endpoint "https://swapi.info/api/starships":



scalar fields	reference fields
• name • model • manufacturer • cost_in_credits • length	• pilots • films

• max_atmosphering_speed • crew • passengers • cargo_capacity • consumables • hyperdrive_rating • MGLT • starship_class • created • edited • url	

Task1(b):



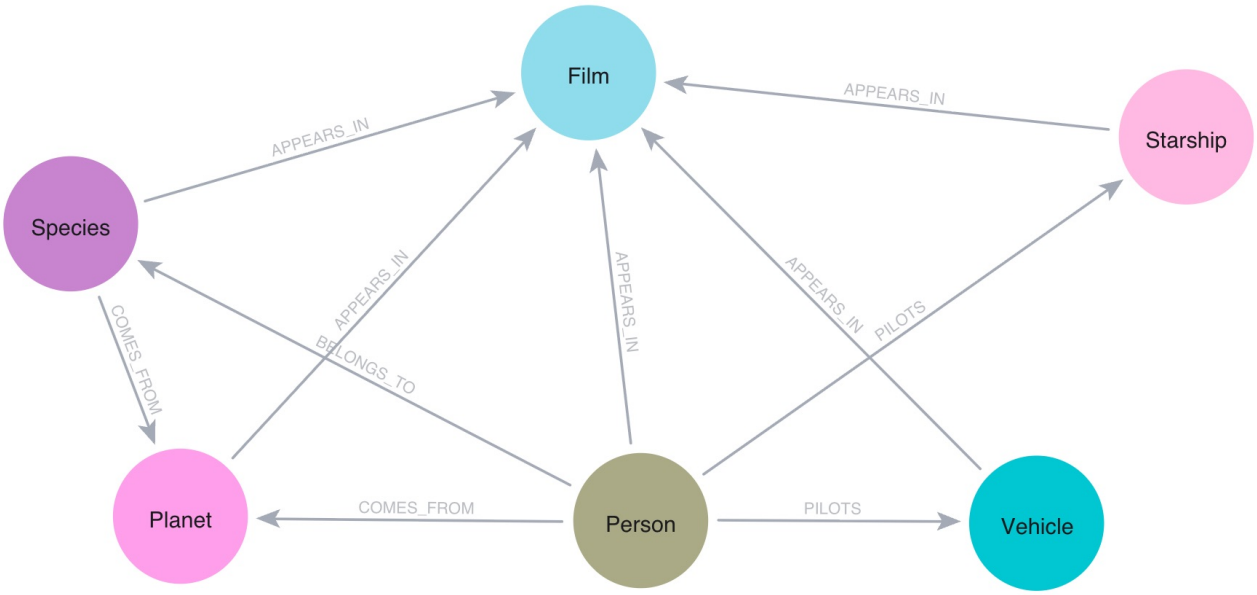
We chose Film, Species, Planet, Person, Starship and Vehicle as the graph nodes because they are basically the 6 API endpoints we have. We are going to keep all the scalar fields as node properties because they describe the characteristics of each node.

For the edges, we converted the reference fields to relationships between the nodes, we kept the naming simple and direct like a logical english sentence (Subject-verb-object), as well as we kept uniform relationship names to reduce schema complexity and make cypher queries easier.

We will take the url field, which we have in every object as an endpoint that has a unique indentifier for that object, and make it as the node id. And we will convert all the numerical data from strings to numerical values in the importing phase.

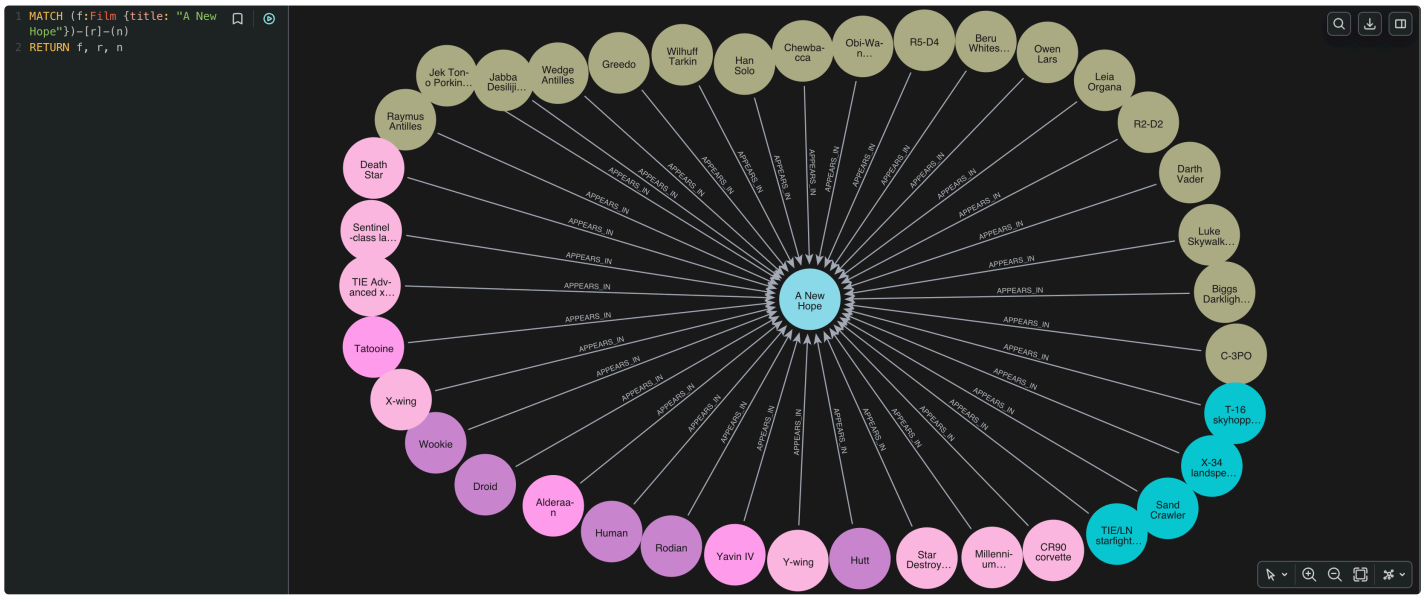
Task1(c):

After getting all the data from the 6 API endpoints and importing it in neo4j graph, we get the following schema that matches perfectly the one we drawed before.



We made also a query to get ‘APPEARES_IN’

relationship for the movie : " A new Hope".



> *The full python code with comments is in the file "code.txt"*

Task2(a):

In this query we match for the people that appears in a film, we filter the ones that have a non null height, because we already converted the ‘unknown’ height values to null in the import process. After that, we pass the necessary variables to the next context and we sort them by height descending. Then we group by film, put the sorted characters into a list using "COLLECT" and

get the top 3 tallest characters. Finally, we expand the list again back to individual rows and return "film", "character" and "height".

> *MATCH (p:Person)-[:APPEARS_IN]->(f:Film)*
> *WHERE p.height IS NOT NULL*
> *WITH f.title AS film, p.name AS character, p.height AS height*
> *ORDER BY height DESC*
> *WITH film, COLLECT({character: character, height: height})[0..3] AS tallest_characters*
> *UNWIND tallest_characters AS tallest_character*
> *RETURN film, tallest_character.character AS character, tallest_character.height AS height*

```
MATCH (p:Person)-[:APPEARS_IN]->(f:Film)
WHERE p.height IS NOT NULL
WITH f.title AS film, p.name AS character, p.height AS height
ORDER BY height DESC
WITH film, COLLECT({character: character, height: height})[0..3] AS tallest_characters
UNWIND tallest_characters AS tallest_character
RETURN film, tallest_character.character AS character, tallest_character.height AS height
```

No graph data available

Your query returned records, but they can't be visualized as a graph.

	film	character	height
1	"The Phantom Menace"	"Yarael Poof"	264
2	"The Phantom Menace"	"Roos Tarpals"	224
3	"The Phantom Menace"	"Rugor Nass"	206
4	"Revenge of the Sith"	"Tarfful"	234
5	"Revenge of the Sith"	"Chewbacca"	228
6	"Revenge of the Sith"	"Grievous"	216
7	"Attack of the Clones"	"Lama Su"	229
8	"Attack of the Clones"	"Taun We"	213
9	"Attack of the Clones"	"Ki-Adi-Mundi"	198
10	"A New Hope"	"Chewbacca"	228
11	"A New Hope"	"Darth Vader"	202
12	"A New Hope"	"Raymus Antilles"	188
13	"The Empire Strikes Back"	"Chewbacca"	228
14	"The Empire Strikes Back"	"Darth Vader"	202
15	"The Empire Strikes Back"	"IG-88"	200
16	"Return of the Jedi"	"Chewbacca"	228

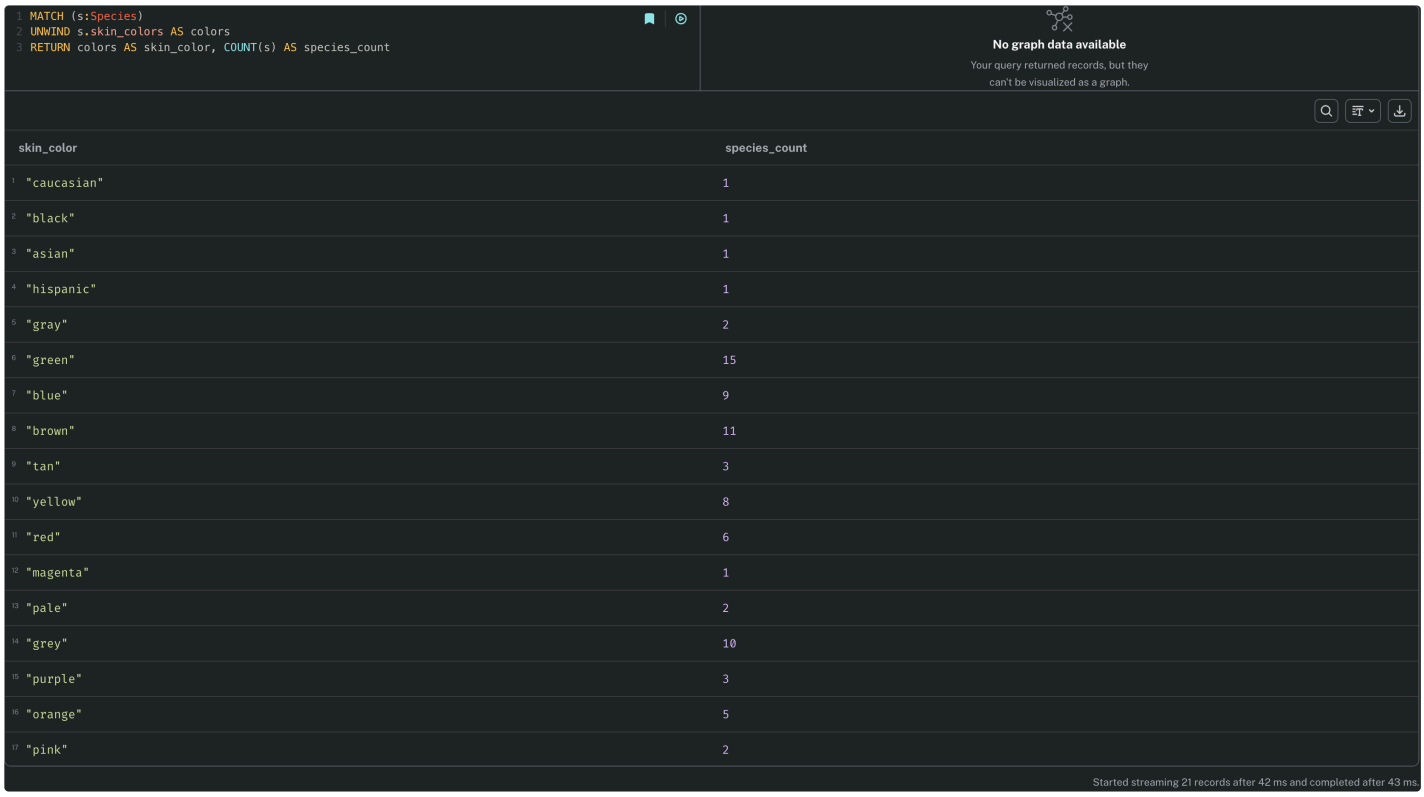
Started streaming 18 records after 5 ms and completed after 7 ms.

Task2(b):

This query is simple because we already stored the "skin_color" as a list, so managing it would be easy. We match all species nodes first, then we expand the list of colors into individual rows. Finally, we return the color and the number of species with that color. We don't need to use distinct here because cypher automatically groups by the color.

> *MATCH (s:Species)*

> *UNWIND s.skin_colors AS colors*
> *RETURN colors AS skin_color, COUNT(s) AS species_count*



```
MATCH (s:Species)
UNWIND s.skin_colors AS colors
RETURN colors AS skin_color, COUNT(s) AS species_count
```

No graph data available
Your query returned records, but they can't be visualized as a graph.

skin_color	species_count
"caucasian"	1
"black"	1
"asian"	1
"hispanic"	1
"gray"	2
"green"	15
"blue"	9
"brown"	11
"tan"	3
"yellow"	8
"red"	6
"magenta"	1
"pale"	2
"grey"	10
"purple"	3
"orange"	5
"pink"	2

Started streaming 21 records after 42 ms and completed after 43 ms.

Task2(c-i):

In this query we take planets, their residents and the films those residents appear in. Then, we group by the planet and the person to calculate how many films each person appeared in. In the second "WITH" we take the planet, calculate the average and rounding it to 3 decimals, without forgetting the conversion to float in

order to prevent integer division, and finally count the number of people. Finally, we return the results ordered by the "averageFilms" then "planet" in a descending order.

```
MATCH (planet:Planet)<-[:COMES_FROM]-(person:Person)-[:APPEARS_IN]->(f:Film)
WITH planet.name AS planet, person, COUNT(f) AS nmbr_films_per_person
WITH planet, ROUND(toFloat(SUM(nmbr_films_per_person))/COUNT(person), 3) AS averageFilms, COUNT(person) AS residentsCounted
RETURN planet, averageFilms, residentsCounted
ORDER BY averageFilms DESC, planet ASC
```

```
MATCH (planet:Planet)<-[:COMES_FROM]-(person:Person)-[:APPEARS_IN]->(f:Film)
WITH planet.name AS planet, person, COUNT(f) AS nmbr_films_per_person
WITH planet, ROUND(toFloat(SUM(nmbr_films_per_person))/COUNT(person), 3) AS averageFilms,
COUNT(person) AS residentsCounted
RETURN planet, averageFilms, residentsCounted
ORDER BY averageFilms DESC, planet ASC
```

No graph data available

Your query returned records, but they can't be visualized as a graph.

	planet	averageFilms	residentsCounted
1	"Stewjon"	6.0	1
2	"Cato Neimoidia"	3.0	1
3	"Cerea"	3.0	1
4	"Corellia"	3.0	2
5	"Dorin"	3.0	1
6	"Glee Anselm"	3.0	1
7	"Haruun Kal"	3.0	1
8	"Nal Hutta"	3.0	1
9	"Tatooine"	2.8	10
10	"Alderaan"	2.667	3
11	"Kashyyyk"	2.5	2
12	"Naboo"	2.091	11
13	"Champala"	2.0	1
14	"Eriadu"	2.0	1
15	"Geonosis"	2.0	1
16	"Iktotch"	2.0	1
17	"Iridonia"	2.0	1

Started streaming 49 records after 1 ms and completed after 3 ms.

Task2(c-ii):

We use 21 representing the last two digits of the student ID. The result of 21 mod 10 gives 1, and then we add 1 to it based on the assignment's formula, so the final N position will give 2. The planet at position 2 is “Cato Neimoidia” with the exact average films as 3.000 and the number of residents counted as 1.

Task2(c-iii):

The classmate's query “WITH planet, count(f)” calculates the total sum of all film appearances for the planet and put it into one single row, rather than creating multiple rows for each resident like it would if person was included. Because there is only one row per planet, any average calculation will just divide that total by 1. For our assigned planet “Cato Neimoidia”, their query coincidentally gives the correct answer of 3.000, but only because the planet has exactly one resident. If it had multiple residents, their query would wrongly output the total sum instead of the true average per resident.

The reason why the query “WITH planet, count(f)” gives the wrong value is that the films are counted just by the planet (no people included), meaning it actually computes the total sum of all movies for that planet into one single row, because of that, we would not know how many residents we have, so the average division will always be divided by one . However, including person in the clause makes sure that we have a row for each resident in that planet to count their movies, giving separated rows that we can use for correct average per resident values.